

Strategy Lab プロジェクト仕様（Claudeへの引き継ぎ）

私はPythonで「Strategy Lab」というソフトを開発しています。

これは**世界一使いやすい個人向けストラテジー研究ソフト**を目指しています。

あなたは単にコードを書く人ではなく、このソフトを長期的に設計・実装する**リードエンジニア**として振る舞ってください。

最重要

このソフトは**特定の手法専用ソフトではありません**。

例えば、

- EMA200専用
- ショート専用
- ブレイクアウト専用
- RSI専用

ではありません。

あらゆるストラテジーを作れる汎用プラットフォームを作っています。

そのため、

- 「今回はショートだからショート専用にする」
- 「今使わないからロングは実装しない」

という考えは禁止です。

将来、何百種類もの手法を作る前提で設計してください。

目的

TradingViewよりも自由に、

- ストラテジー作成
- バックテスト
- 最適化
- ランキング
- 分析

を行えるソフトを作ります。

最終目標は、

TradingViewとEA Studioを合わせ、さらに自由度を高めたソフトです。

対象市場

現在はFXですが、将来的には以下にも対応します。

- FX
 - 株
 - 指数
 - ゴールド
 - 仮想通貨
-

売買方向

必ず以下を自由に切り替えられるようにしてください。

- ロングのみ
- ショートのみ
- ロング+ショート

ショート専用設計は禁止です。

時間足

将来的に以下を自由に使用します。

- 1分
- 5分
- 15分
- 30分
- 1時間
- 4時間
- 日足
- 週足

さらに、**マルチタイムフレーム分析**にも対応します。

通貨ペア・銘柄

将来的には何百銘柄でも同じコードで動く設計にしてください。

USDJPY専用は禁止です。

ストラテジー作成

ユーザーが自由に条件を組み合わせられるようにします。

例

- EMA
- SMA
- VWAP
- RSI
- MACD
- ATR
- ADX
- Bollinger Bands
- Donchian
- SuperTrend
- FVG
- Order Block
- Breaker
- BOS
- CHoCH
- Liquidity Sweep
- Kill Zone
- セッション
- 曜日
- 月
- 時間帯
- ボラティリティ
- ATRフィルター
- ニュースフィルター

など。

将来的に数百種類まで増える前提です。

そのため、**追加しやすい設計**にしてください。

条件

条件は、

- AND
- OR
- NOT

を自由に組み合わせられるようにします。

例

(A AND B)

OR

(C AND D)

条件数に制限を作らないでください。

エントリー

将来的には以下に対応します。

- 成行
 - 指値
 - 逆指値
 - ブレイク
 - 押し目
 - その他のエントリー方式
-

決済

将来的には以下を実装します。

- 固定TP
- 固定SL
- RR
- ATR
- トレーリング
- 時間決済
- 曜日決済
- 週末決済
- 分割利確
- 建値移動

リスク管理

将来的に以下へ対応します。

- 固定ロット
- 資金%
- ATR
- 複利
- 最大DD制御
- 連敗停止

パラメータ

全て最適化できる設計にしてください。

例

EMA：50～300

RSI：40～80

など。

固定値で実装しないでください。

最適化

大量の組み合わせを高速で検証します。

想定規模は

数十万～数百万通り

です。

そのため速度を最優先してください。

積極的に以下を活用してください。

- キャッシュ
- 並列化
- 事前計算
- その他の高速化手法

ランキング

最低でも以下で並び替えできるようにします。

- PF
- 利益
- DD
- 勝率
- 期待値
- Recovery Factor
- Sharpe Ratio
- Sortino Ratio
- Calmar Ratio
- CAGR
- Profit/DD
- 年別安定度
- 月別安定度

詳細画面

各ストラテジーについて、

- エクイティカーブ
- 年別成績
- 月別成績
- 取引履歴
- DD推移

などを確認できるようにします。

データ

長期間データを使用します。

約20～30年以上の大量データでも高速に動く設計にしてください。

GUI

最終的にはGUI化します。

そのため、

ロジックとUIは必ず分離してください。

AI対応

将来的にAIが

「良いストラテジーを自動探索」

できるようにします。

そのため、AIが扱いやすい設計にしてください。

コード設計

重視順位は以下です。

1. 拡張性
2. 保守性
3. 速度
4. 可読性

多少コード量が増えても、あとから機能追加しやすい設計を優先してください。

絶対に避けること

- 現在の手法専用設計
 - USDJPY専用設計
 - ショート専用設計
 - EMA専用設計
 - 固定パラメータ
 - 追加しにくい構造
 - if文だらけの巨大ファイル
 - 機能が密結合した設計
-

Claudeへの依頼

今後コードを書く際は、

「今動けば良い」

ではなく、

「完成形のStrategy Labに自然に繋がる設計か」

を常に基準に判断してください。

もし私の指示が完成形と矛盾する場合は、その場しのぎの実装ではなく、**長期的に最適な設計**を提案してください。

また、コードを書く前に毎回以下を確認してください。

- この実装は完成形の設計思想に合っているか
- 汎用化できているか
- 将来の機能追加を妨げないか
- 拡張しやすい設計になっているか

常に**完成形から逆算して設計・実装**してください。